

Assignment 2 (Group Assignment): Amazone

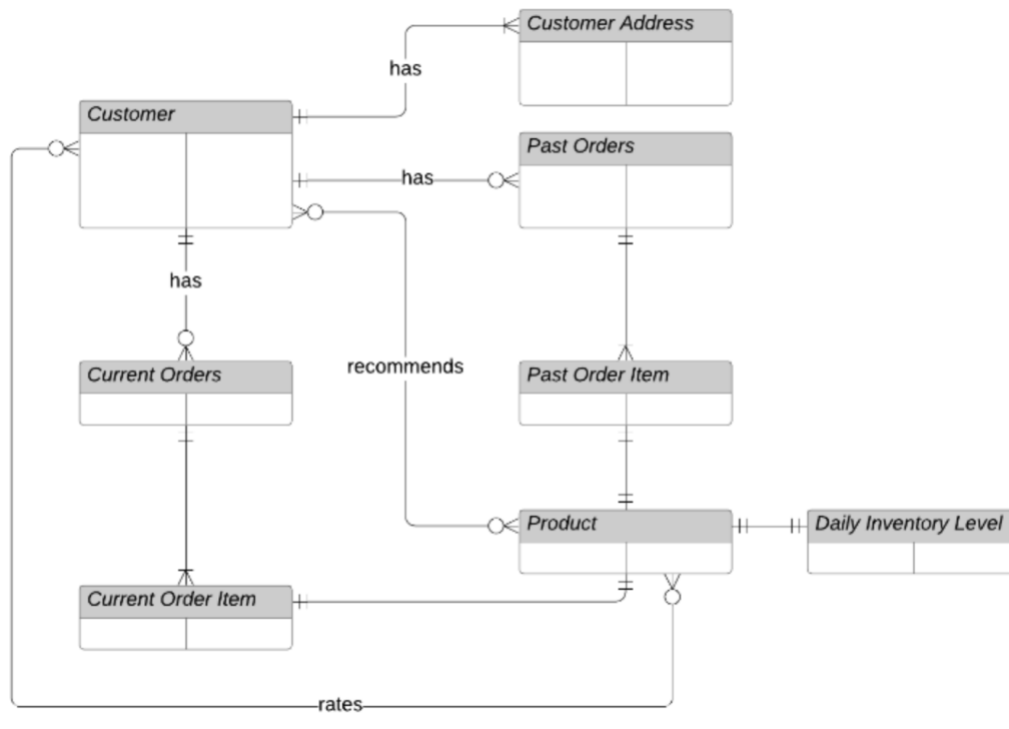
Your group's task is to design, implement and demonstrate a NoSQL database for the Amazone online shopping website that operates only in the UK. Each group will submit presentation slides co-design by the group members, a recorded video presentation that is between 15 minutes to 20 minutes long (each member of the group must present a minimum of one slide), and a zipped copy of their final database and queries. Each group member will also submit a personal reflective essay. If you find any ambiguities in the specification of this assignment, you should decide what to do and clearly document and justify your decisions in your group slides and presentation.

Case Background

Amazone has decided to expand its operations by delivering fresh groceries. Amazone partnered with Morrizon, a UK grocery retailer, to offer same-day and instant grocery delivery service. Amazone has decided to experiment with same-day and instant grocery delivery using Manchester. Six Morrizon grocery stores have been chosen for instant pick-up and delivery by delivery drivers who are Amazon partners. The Amazone team has contacted your team to develop a new online platform that accommodates both the existing and new business models. Your team has received the following information to help with the data modelling and implementation service.

ER Diagram

Below is an ER diagram of the old database schema. However, your team has been informed that this schema may be slightly different from the final schema. Hence, you are free to modify the schema if you wish while integrating the new data requirements for the new business. Your group presentation should include a final schema diagram in the form of a Collection Relationship Diagram that accommodates the new business model and data requirements.



Requirements, Assumptions, Projections

1. Customer contains attributes such as name, gender, age, customer ID, address, etc. We assume that the customer information is relatively stable. The number of customers is huge (in millions).
2. Customer Address has attributes such as house number, street, city, postcode/zip code, etc. A customer can have more than one address (such as billing address(es), shipping address(es), etc.). We assume that the addresses are relatively stable.
3. Customers may have Past Orders, each comprising an order ID and the total order cost. Each order is linked with a list of ordered products represented by Past Order Items, where each item has an attribute giving its quantity in its order. Each ordered item is linked with one product in Product. We assume that the number of past orders can be large for individual customers (say 200 orders) and is rarely queried.
4. Current Orders have a similar structure to Past Orders. We assume that the number of current orders of a customer is small (say 5-10 orders), but this information is frequently retrieved.
5. We assume that there are two product segments, Fresh and Others. Fresh products are groceries, which must be picked and delivered from the nearby Morrison grocery store. Hence, each fresh product has attributes that associate it with a store. Each Store has attributes such as an address, location (latitude and longitude), list of grocery items available for ordering and pickup, etc.
6. All fresh orders are picked up by Amazon partners (delivery drivers) registered as Partners. Partners' attributes are similar to customer attributes except that we need to know their present location if they are idle or active, and if they are active, we need to know if they are on a delivery errand. These attributes will help the application to decide which partner to assign a pickup and delivery task. Also, we may want to know

the details of products delivered and some statistics to calculate pay-outs to each partner – you are to decide these statistics.

7. In a real-world system, there may be thousands of Fresh Product categories, but Amazone has decided to start with three Fresh Product categories, which are bakery, drinks, and fruits and vegetables. Each product type in each of the fresh categories has the following attributes:
 1. product name
 2. short description At least 2 queries indicating a customer ordering a fresh product. The query should include the assignment of pickup and delivery tasks to a partner based on location parameters. The query should return, e.g., details of the product ordered, delivery partner location and/ or ETA, and details of the delivery partner – name and ratings (if available).
 3. product dimensions
 4. product weight or quantity (for liquids)
 5. expiry date
 6. country of origin
 7. average customer rating score
 8. standard price to customers
 9. cost of products from Morrizon (i.e., the cost to Amazone)
8. In a real-world system, there may be thousands of product types for Other Products. Here, we assume that there are four types: books, CDs, mobile phones, and home appliances. The number of products is huge (millions). Each product type has the following attributes:
 1. product name
 2. short description
 3. product dimensions
 4. shipping weight
 5. average customer rating score
 6. standard price to the customer
 7. cost of products from the supplier (i.e. cost to Amazone)
9. Additionally, each product type has its specific attributes:
 1. Book has:
 1. author name
 2. publisher
 3. year of publication
 4. ISBN
 - 5.
 2. CD has:
 1. artist name
 2. number of tracks
 3. total playing time
 4. publisher
 3. Mobile phone has:
 1. brand
 2. model
 3. colour
 4. features
 4. The home appliance has:
 1. colour

2. voltage
 3. style
10. Each product is linked with its Daily Inventory Level for the past five years, with attributes
 1. date
 2. inventory quantity
 3. storage warehouse location
 4. storage warehouse name
11. A customer can give a rating to a product, but not every customer will rate every product. In fact, the rating is typically very sparse. For example, for each product, perhaps only 1% of the customers have ever rated it, and for each customer, they have only rated about 0.1% of the products. The rating information needs to be intensively retrieved to calculate predicted ratings for products a customer hasn't bought yet and will be used to derive a set of recommended products for each customer. Both the rating and the recommended products are frequently updated. The recommended products must be displayed quickly every time a customer logs in to Amazone.

Assignment Tasks

Your group must first decide how to split the project into sub-tasks between its members, and your presentation slide must contain a list of members and sub-tasks assigned to each member. Each group should choose one member to be the coordinator, who will be responsible for submitting all the assignment deliverables.

Task 1: Design a NOSQL schema for Amazone

Use the format below (Collection Relationship) to describe and present your final schema, and include this in your group presentation slide:

```

COLLECTION1 {
field1: type                                // for single valued fields
field2: {
    sub-field1: type                        // for embedded fields
    sub-field2: type
    ... etc
}
field3: [ type ]                            // for arrays
field4: ref<COLLECTION.Field>              // for referencing fields
...etc
} // end of COLLECTION1

COLLECTION2 {
... etc
} // end of COLLECTION2

... etc

```

Task 2: Implement a Demonstration Database

Based on your schema from Task 1, implement a demonstration database using MongoDB Compass or MongoDB Shell. The specific contents (such as the customer names, addresses, product names, etc) are for you to choose, but the following 10 requirements must be satisfied:

1. at least 20 customers.
2. at least 10 product samples for each product (book, CD, mobile phone, and home appliance).
3. at least 5 fresh products from each fresh product category.
4. at least 5 store locations for fresh products pickup and delivery
5. at least 5 partners for instant pickup and delivery
6. each customer has at least 2 current orders and at least 5 past orders.
7. each customer has rated at least 3 products; at least 2 customers have rated each product.
8. recommend each customer with at least 2 products.
9. only logs for 2 days of daily inventory levels are needed for each product.
10. at least one of the collections should be indexed for an efficient query. You are to decide which of the collections the index should be created and on which field.

You should implement at least 10 queries designed by yourselves using MongoDB Compass, MongoDB Shell, Python or NodeJS to demonstrate that your database works. You should implement the queries as follows:

1. At least 2 queries indicating a customer ordering a fresh product. The query should include the assignment of pickup and delivery tasks to a partner based on location parameters. The query should return, e.g., details of the product ordered, delivery partner location and/ or ETA, and details of the delivery partner – name and ratings (if available).
2. At least 1 query that indicates a user searching for available fresh products. The products should be displayed based on the user's location.
3. At least 2 queries that indicate a customer ordering a product, adding it to the cart and making payment.
4. At least 2 queries indicating a manager checking sales and inventory performance. Where possible, the query result should be visualised using Pandas (table) or Matplotlib (charts).
5. The remaining queries should be designed by yourself. The designed queries must be realistic, that is, they should be queries that apply to an ecommerce application. At least 30% of these queries must use the aggregation pipeline

Export and collate all your collections in JSON format (export from MongoDB Compass to JSON files), including query commands and the corresponding query results (by copying and pasting from MongoDB Compass, MongoDB Shell, Python, or NodeJS).

Deliverables

There are four deliverables:

- One presentation slide per group
- one recorded video presentation per group
- one zipped copy of collections, queries, and results per group
- one report **per individual student**